# TODO: WORK IN PROGRESS

**Bachelor's Thesis**

by

*Jakob Meyerhöfer*

*(Mail: meyerhoe@rptu.de)*

March 10, 2026

University of Kaiserslautern-Landau
Department of Computer Science
67663 Kaiserslautern
Germany

Examiner: Prof. Dr. Ralf Hinze
Cass Alexandru (Gregor Cassian Alexandru), M.Sc.

## Abstract

**Marker**

*TODO*

## Summary

**Marker**

*TODO*

# Contents


**1 Introduction** **1**
- 1.1 Problem Statement and Objectives . . . 2
- 1.2 Research Questions . . . 3
- 1.3 Thesis Structure . . . 3

**2 Background and Related Work** **5**
- 2.1 Challenges for Novice Programmers . . . 5
- 2.2 Error Messages and Programming Education . . . 5
- 2.3 Improving Feedback: Existing Aproches (Rust?) . . . 5

**3 Problem Analysis** **7**
- 3.1 Context and Assumptions . . . 7
- 3.2 Observed Issues in Existing Compiler/Test Output . . . 8
  - 3.2.1 Examples of current error messages . . . 8
  - 3.2.2 Recurring structural problems . . . 9
  - 3.2.3 Issue 1: Essential information is fragmented or implicit . . . 10
  - 3.2.4 Issue 2: Mixing core mismatch information with framework internals . . . 10
  - 3.2.5 Issue 3: Inconsistent structure across test types and frameworks . . . 11
  - 3.2.6 Issue 4: Property-based testing output is informative but not pedagogically structured . . . 11
  - 3.2.7 Issue 5: Missing execution context for stateful or multi-step scenarios . . . 11
  - 3.2.8 Mock-up: additional observations to validate with literature (not yet backed by your local examples) . . . 12
- 3.3 Derived Requirements and Design Implications . . . 12
  - 3.3.1 Requirements . . . 12
  - 3.3.2 Design implications for Assertify/Checkify . . . 13
  - 3.3.3 Completion Checklist for This Chapter . . . 14

**4 Assertify (and Checkify): Design and Implementation** **15**
- 4.1 Design Goals and Principles . . . 15
- 4.2 Output Model: Representing Tests . . . 16
- 4.3 Mechanisms: Expression Capture and Value Extraction . . . 17
  - 4.3.1 Expression capture via code quotations . . . 18
  - 4.3.2 Value extraction for unit tests and property based tests . . . 20
- 4.4 Reporting: Formatting and Integration with the Test Pipeline . . . 24
  - 4.4.1 Integration with example-based unit tests . . . 24
  - 4.4.2 Integration with property-based tests . . . 25

4.4.3 Formatting strategy . . . . . 25
4.4.4 Configurable verbosity and optional context . . . . . 25

**5 Results and Discussion** **27**
5.1 Research Question Coverage and Requirement Satisfaction . . . . 27
5.2 Artifact Outcomes . . . . . 27
5.3 Key Findings and Interpretations . . . . . 27
5.4 Strengths and Limitations . . . . . 27

**6 Conclusion** **29**
6.1 Summary . . . . . 29
6.2 Future Work . . . . . 29
6.3 Final Remarks . . . . . 29

**List of Figures** **31**

**List of Listings** **33**

**List of Tables** **35**

**My Code** **37**

**My Ideas** **39**

**Bibliography** **41**

# 1 Introduction

**Marker**

*IMPROVE*

All programmers have to start at the beginning: What is programming? What is required to write and execute code? What are variables and methods and how are they used? From there, learners gradually encounter key concepts and algorithms, and later data structures such as maps and sets. However, moving from basic syntax to independently writing correct programs is a long process, which also includes implementing corresponding unit tests to validate solutions.

Understanding and interpreting error messages is a fundamental challenge for novice programmers. Error messages serve as an essential form of feedback, enabling programmers to diagnose issues in their code. In practice, standard diagnostics - especially those produced by compilers and testing frameworks - are often technical, verbose, and difficult for beginners to interpret.

This becomes particularly apparent in introductory programming courses where students receive exercise templates with predefined tests and are expected to implement missing functions or data structures. In such settings, learners are frequently unfamiliar with unit testing and are immediately confronted with stack traces and framework-specific output formats.

In F#, multiple testing frameworks and assertion libraries are available. A commonly used option is Microsoft.VisualStudio.TestTools.UnitTesting. For example, using `Assert.AreEqual` can yield the following output on failure:

```
Assert.AreEqual failed. Expected:<[1; 2; 4]>. Actual:<[1; 4; 2]>.
    at MapTests.TestsMapSortedList.Tests.testSetBeispiele() in /code/
    ↪ TestsMapSortedList.fs:line 53
    at System.RuntimeMethodHandle.InvokeMethod(Object target, Void**
    ↪ arguments, Signature sig, Boolean isConstructor)
    at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object obj,
    ↪ BindingFlags invokeAttr)
```

**Error Message 1.1** Example Ouput of `Assert.AreEqual`

For more advanced programmers, this output is usually sufficient: they can quickly identify the relevant mismatch and ignore the remaining details. See Section TODO for a discussion of common issues in diagnostic messages and strategies for extracting key information.

In my experience as a tutor for a first-semester lecture, I frequently observe students struggling with such messages, unsure what went wrong and how to fix their code. Instead of guiding them toward a solution, poorly structured error messages

can lead to frustration and trial-and-error debugging, which may hinder learning and delay progress on key programming concepts.

Didactically, complex concepts are typically introduced gradually, often by reducing incidental complexity at the beginning:

- Lightweight IDE's are often preferred to heavy, feature-rich IDE's
- Simplistic I/O is introduced before full input validation only valid inputs and error handling (e.g., malformed inputs, exceptions, and missing values).

The same Diagnostic verbosity can be staged for testing: Initially showing only the most relevant information and revealing additional technical details as students gain experience. This thesis explores how a similar scaffolding approach can be applied to testing feedback. By providing more minimalistic and structured output, the goal is to help students interpret failures more effectively and support a more efficient and engaging learning process.

## 1.1 Problem Statement and Objectives

**Marker**

*IMPROVE*

The primary problem addressed in this thesis is that, in introductory programming settings, the feedback produced bycommon testing frameworks is often not presented in a form that is immediately interpretable for novice programmers. In particular, failure reports frequently combine the core mismatch information (i.e., what was expected versus what occurred) with framework-specific detail such as stack traces and internal method calls. As a consequence, beginners may struggle to identify the relevant part of the output and translate it into concrete corrective actions in their own code.

From a technical perspective, this raises the question of how test feedback can be reformatted such that it is **(i)** consistent and cognitively simple for first-semester students, while **(ii)** remaining compatible with established testing infrastructures. This thesis therefore investigates the design and implementation of a lightweight assertion and reporting layer, *Assertify*, which produces a compact and structured summary of test failures.

The objectives of this thesis are as follows:

- Analyse the structure and typical content of failing test output in beginner exercises, with particular attention to which elements are essential for interpreting the failure.
- Design and implement a custom test/assertion library that enforces a consistent reporting scheme (e.g., a compact summary of the `tested expression` and `expected` as well as `actual` value) on top of existing frameworks.
- Evaluate the resulting artifact with respect to the derived requirements and reflect on remaining limitations, thereby identifying technically feasible extensions and future improvements.

To achieve these objectives, the thesis presents *Assertify* as a wrapper around existing mechanisms for unit and property-based testing and discusses how the reporting format can be generalised, extended, and integrated into introductory teaching workflows.

## 1.2 Research Questions

**Marker**

*IMPROVE*

This thesis seeks to answer the following research questions:

**RQ1 (Problem characterization)** Which structural characteristics of compiler and testing-framework outputs make failing tests difficult to interpret for novice programmers in introductory programming exercises?

**RQ2 (Design requirements)** Which requirements can be derived for novice-oriented test feedback with respect to consistency, cognitive load, and the actionability of failure information?

**RQ3 (Reporting scheme)** How can a uniform *tested expression/expected/actual* reporting scheme be defined such that it remains applicable across different kinds of tests (example-based unit tests and property-based tests)?

**RQ4 (Technical feasibility)** How can this reporting scheme be implemented in F# using Quotations and Unquote for expression capture and evaluation, while remaining compatible with existing tooling through wrappers around `Microsoft.VisualStudio.TestTools.UnitTesting` and `FsCheck`?

**RQ5 (Results and discussion)** To what extent does the implemented artifact (*Assertify* & *Checkify*) satisfy the derived requirements in practice, and what limitations and trade-offs emerge from the wrapper-based approach with respect to expression coverage, supported assertion forms, formatting constraints, and integration boundaries?

By addressing these questions, this thesis positions its contribution as the design and implementation of a test feedback layer that simplifies and standardizes failure reporting for novice programmers, and as a critical reflection on how such a library can be extended and integrated into teaching contexts.

## 1.3 Thesis Structure

**Marker**

*IMPROVE*

The structure of this thesis is as follows:

**Background and Related Work** reviews research on the difficulties novice programmers face when interpreting compiler and test diagnostics, and summarises prior approaches that aim to improve the structure and usefulness of error feedback. Building on these perspectives, the chapter motivates the thesis goal of presenting failures in a more consistent and cognitively simple format.

**Problem Analysis** examines typical compiler and testing-framework outputs as they appear in beginner exercises and characterises their technical properties, such as verbosity, inconsistency, and the mixing of essential information with framework internals. Based on these observations, the chapter derives design requirements for improved test failure reporting, with a stable *tested expression/expected/actual* summary as the primary user-facing interface.

**??** presents the design rationale and implementation of *Assertify* as a lightweight test and assertion library that wraps existing testing infrastructures. It explains how a consistent `expected=actual` output format is produced, including the use of Quotations and Unquote for expression capture and evaluation, as well as wrappers around `Microsoft.VisualStudio.TestTools.UnitTesting` and `FsCheck`. The chapter also discusses implementation choices and their implications for extensibility.

**Results and Discussion** reports the results of the implemented approach and discusses them in relation to the derived requirements and research questions. The discussion focuses primarily on technical and qualitative properties achieved by the artifact (e.g., output consistency, coverage of supported assertion forms, and integration with unit and property-based testing workflows) and uses illustrative comparisons to baseline framework output to highlight differences in interpretability, while separating concrete observations from generalisable claims.

**Conclusion** concludes the thesis by summarising the contributions and revisiting the research questions in light of the presented results. It then outlines future work, including potential extensions within the wrapper-based approach (e.g., broader expression coverage, richer formatting layers, and tighter tooling integration) and longer-term directions that would require more substantial redesign beyond wrappers.

# 2 Background and Related Work

**Marker**

*TODO*

## 2.1 Challenges for Novice Programmers

**Marker**

*TODO*

## 2.2 Error Messages and Programming Education

**Marker**

*TODO*

## 2.3 Improving Feedback: Existing Aproches (Rust?)

**Marker**

*TODO*

# 3 Problem Analysis

**Marker**

*IMPROVE*

Baseline compiler and test outputs are a central feedback channel in introductory programming courses, yet they often fail to support novices in turning failures into actionable debugging steps. The problem is typically not that messages contain no relevant information, but that crucial facts are difficult to find, inconsistently structured, or weakly connected to the learner's own code. As a result, students must spend effort on reconstructing what was actually evaluated and why it failed instead of focusing on correcting their implementation.

Prior work indicates that error messages are frequently perceived as confusing and difficult to act upon, particularly for beginners, and that difficulties in reading error messages are associated with worse performance on programming tasks [3], [2]. Moreover, improving error messages is constrained by technical challenges such as locality and the mapping from internal tool representations back to source-level explanations [3]. These observations motivate a systematic analysis of baseline outputs and the derivation of explicit requirements for novice-oriented test reporting.

## 3.1 Context and Assumptions

**Marker**

*IMPROVE*

The analysis in this chapter is situated in introductory programming exercises where students implement functions according to a specification and receive feedback through compilation and automated tests. Students typically iterate by running tests locally or by submitting to an assessment system then inspecting the returned compiler and test feedback. A key assumption is that students primarily control the *solution code*, while the tests may be partially or fully hidden. Consequently, the error message becomes the primary artifact from which learners must derive actionable debugging information.

The examples in this thesis focus on the F# ecosystem and a test setup that combines (i) example-based tests using Unquote and (ii) unit and property-based tests executed via the MSTest framework and FsCheck. The analysis assumes that novice students have limited familiarity with testing conventions such as stack traces, reflection-based invocation, and framework-specific formatting. They may also struggle to distinguish between information that is actionable for correcting

their own logic and information that is mainly relevant for diagnosing the test infrastructure.

Finally, the chapter assumes that effective feedback should reduce reconstruction effort. Barik et al. show that developers spend substantial time attending to error message regions and that difficulty in reading error messages predicts task performance [2]. While their participants are not restricted to novices, the findings motivate presenting relevant failure information in a structured, easy-to-scan form.

## 3.2 Observed Issues in Existing Compiler/Test Output

**Marker**

*IMPROVE*

This section identifies recurring issues in baseline outputs produced by common compiler and testing tools. The focus is on how test failures are reported and how students must reconstruct key information that is not explicitly presented. The examples are drawn from representative beginner exercises used throughout this thesis.

### 3.2.1 Examples of current error messages

Error messages often require the programmer to understand not only *what* was tested but also *how* it was tested. For instance, tests for data structures and stateful components may involve sequences of operations (e.g., insertions and removals) followed by an invariant check. If a failure occurs after multiple steps, baseline output rarely describes the sequence leading to the assertion point, even though a compact description of the minimal failing history would often be sufficient for debugging.

In addition, baseline frameworks frequently present essential information in a fragmented manner. For example, `Assert.AreEqual` can report an expected/actual mismatch, but it does not inherently encode the tested expression. Unless the test method name and surrounding context are highly descriptive, students may struggle to associate the mismatch with a concrete call in their own code. As a consequence, learners must infer missing context from indirect cues such as test names, inputs printed elsewhere, or stack traces.

#### Example 1 — `digitSum`

Consider the exercise `digitSum : Nat → Nat`, where the task is to compute the digit sum of a natural number. A representative novice mistake is an incorrect base case, such as returning `1N` for input `0N`. Example-based tests using Unquote can display the failed equality as a quoted expression and show the boolean result of the comparison, but the output also contains repeated, qualified expression renderings and a stack trace that dominates the overall message.

For novices, this reduces the signal-to-noise ratio: the crucial mismatch is present, yet not highlighted as a compact summary.

When property-based testing is used (e.g., FsCheck integrated with MSTest assertions), the output typically reports that a property is falsifiable and provides an original and a shrunk counterexample (often a minimal input such as `0N`). This information is valuable, but the message does not necessarily provide a direct, novice-friendly statement of the failing behaviour in terms of a single reconstructed expression such as:

*Tested expression: `digitSum 0N`. Expected: `0N`. Actual: `1N`.*

Instead, students must combine fragments from different parts of the output (counterexample, expected/actual mismatch, and implicit knowledge about what the test invokes) to arrive at that statement.

**Example 2 — `sortedDigits`**

A similar pattern occurs for boolean-valued exercises such as `sortedDigits : Nat → bool`. Failures may hinge on subtle predicate choices (e.g., strict vs. non-strict comparisons), where the difference between < and <= becomes visible only on specific counterexamples (e.g., repeated digits). In baseline outputs, the essential information is the concrete failing input together with the expected and actual boolean result. However, this information is often presented without an explicit reconstruction of the tested call, and it is not consistently formatted across different testing mechanisms [2]. As a result, students must infer which invocation produced the mismatch and why the counterexample is relevant to the specification.

### 3.2.2 Recurring structural problems

Across the examples above, the following structural problems recur.

**Missing explicit linkage to the tested expression.** Many assertion messages focus on the mismatch (expected vs. actual) without explicitly reporting the expression or call that produced it. For novices, this disconnect is critical because it introduces ambiguity about what exactly was evaluated. The problem is amplified when tests are hidden, since the student cannot inspect the failing test case and must rely on the message alone.

**Fragmentation of essential information.** Even when all necessary facts are present (inputs, expected value, actual value), they may appear in separate lines or in different parts of the output. Students are required to reconstruct the failing behaviour by manually combining fragments. This reconstruction increases cognitive load and encourages trial-and-error rather than systematic debugging.

**Inconsistent message formats across tools.** In the considered setting, example-based tests (e.g., Unquote) and framework assertions (e.g., MSTest `Assert.AreEqual`) produce different output structures and emphasize different aspects of a failure. Such inconsistency prevents novices from developing a stable reading strategy. More generally, diagnostics often originate from internal tool representations that do not map cleanly back to what the programmer wrote [3].

**Verbosity and low-level noise.** Stack traces, framework invocation details, and module-qualified names can obscure the central mismatch. While such details may help experienced developers diagnose infrastructure issues, they often do not help novices fix solution logic. Verbose output can reduce readability and make it difficult to locate actionable information.

**Locality and causality gaps.** Error reports are not always local to the underlying cause. Becker et al. note that errors are frequently detected far away from their generation sites (the "locality problem"), and that diagnostics may reflect tool-internal viewpoints rather than the programmer's conceptual model [3]. In educational contexts, this manifests when the message points to the test harness location (which students may not access) rather than to the relevant part of their own code.

This section characterises the structural properties of baseline outputs that contribute to poor interpretability for novices. The analysis is organised around recurring patterns, illustrated through representative examples from beginner exercises.

### 3.2.3 Issue 1: Essential information is fragmented or implicit

In many baseline test outputs, the information required to reconstruct the failing behaviour is not presented as a single, explicit summary. Instead, learners must infer it by combining multiple fragments (e.g., a test name, a partially printed expression, and a comparison line). When the *tested expression* is not explicitly shown, students may not be able to associate the failure with a specific function call in their submission, especially when a test method executes multiple assertions or multiple inputs.

### 3.2.4 Issue 2: Mixing core mismatch information with framework internals

Failure reports frequently interleave the core mismatch (expected vs. actual) with verbose framework-specific content such as stack traces, internal runner frames, and reflection invocation details. While such information can be valuable for expert debugging, it often dominates the output visually and makes it difficult for novices to identify the actionable part of the report.

### 3.2.5 Issue 3: Inconsistent structure across test types and frameworks

Even within a single course setup, students may encounter multiple output formats depending on whether a failure stems from:

(a) a unit-test assertion,

(b) a quotation-based assertion mechanism, or

(c) a property-based test with counterexample generation and shrinking.

If these formats differ substantially in vocabulary, ordering, and level of detail, students cannot form stable expectations about where to look for essential information.

### 3.2.6 Issue 4: Property-based testing output is informative but not pedagogically structured

Property-based testing frameworks can provide powerful information (e.g., a counterexample input and a shrunk minimal failing case). However, this information is often embedded in output that assumes familiarity with the property-testing workflow. In particular, the relationship between (i) the generated input, (ii) the shrunk counterexample, and (iii) the failing assertion is not always made explicit in a way that supports novice mental models.

### 3.2.7 Issue 5: Missing execution context for stateful or multi-step scenarios

For exercises involving stateful data structures or sequences of operations, a single expected/actual mismatch may be insufficient to understand the failure. If the failure occurs after a sequence of updates, students may need to know *what was executed before the failing check* in order to reproduce the bug. Baseline outputs typically do not preserve such operational context.

**Checkpoint (examples and evidence to add):**

- TODO: Insert 2–3 concrete baseline outputs from your own exercise set and annotate them (highlight: where is the tested expression? where is expected/actual? what is noise?).
- TODO: Add at least one example for each: (i) example-based unit test failure, (ii) quotation/assertion failure, (iii) property-based failure with shrinking.
- TODO: Provide a short "signal vs. noise" breakdown for each example (e.g., mark lines as essential vs. auxiliary).
- TODO: If you discuss multi-step/stateful tests, add a small scenario (e.g., linked list operations) and show precisely what context is missing in baseline output.

### 3.2.8 Mock-up: additional observations to validate with literature (not yet backed by your local examples)

The following patterns are frequently discussed in the broader literature on novice-facing diagnostics and may apply to your setting, but should be validated with either (i) your own collected outputs or (ii) clearly cited prior work:

- TODO: **Jargon and modality switching:** natural-language fragments mixed with code tokens can force frequent context switches during reading.
- TODO: **Redundant repetition:** repeating the same information in multiple forms (e.g., multiple representations of the same expression) can increase reading effort without improving comprehension.
- TODO: **Poor localisation:** messages may be accurate but point novices to symptoms rather than the likely source of misunderstanding.

## 3.3 Derived Requirements and Design Implications

**Marker**

*IMPROVE*

Based on the issues above, this section derives requirements for novice-oriented test failure reporting. Each requirement is phrased as a constraint on the *user-facing report*, independent of any specific implementation technique. The subsequent design chapter then maps these requirements onto concrete mechanisms (e.g., expression capture, value extraction, and framework wrappers).

### 3.3.1 Requirements

**R1: Stable summary structure.** Every failing test should provide a compact, consistently ordered summary containing at least: (i) the tested expression (or an equivalent description of the behaviour under test), (ii) the expected value/condition, and (iii) the actual value/observed result.

**R2: Explicit tested expression.** The report must explicitly identify *what was executed* (e.g., a reconstructed function call) rather than relying on the test name or on implicit context.

**R3: Expected vs. actual as the primary comparison.** The expected and actual outcomes must be presented side-by-side (or in a tightly coupled pair) in a way that supports rapid visual comparison.

**R4: Separation of essential and auxiliary information.** Framework internals (stack traces, runner frames, reflection calls) must not obscure the essential summary. If included at all, such details should be clearly separated and optionally suppressible.

**R5: Applicability across test paradigms.** The same summary structure should apply to both example-based unit tests and property-based tests, even when additional information (e.g., counterexamples) is present.

**R6: Structured representation of property-based counterexamples.** If a property fails, the report must clearly indicate (i) the failing input, (ii) any shrunk/minimised counterexample, and (iii) how this input relates to the tested expression and observed mismatch.

**R7: Support for contextual history when needed.** For multi-step/stateful scenarios, the reporting layer should be able to attach an execution history (e.g., a sequence of operations) to make the failure reproducible and interpretable.

### 3.3.2 Design implications for Assertify/Checkify

The requirements above imply that a reporting layer must (a) gain access to the tested expression at runtime, (b) evaluate it to obtain the actual value, (c) retain or compute the expected value, and (d) serialise both into a stable output model that is independent of any particular baseline framework.

In the scope of this thesis, these implications motivate:

- **Expression capture:** obtain a representation of the tested expression that can be displayed to the user (and simplified where appropriate).
- **Value extraction:** evaluate expressions in a controlled way to compute actual values and to render values consistently.
- **Wrapper-based integration:** intercept failures from existing unit testing and property-based testing frameworks and replace or augment their output without requiring changes to the surrounding tooling.
- **Unified output model:** define a single report representation that can be produced for different assertion forms and test paradigms.

**Checkpoint (traceability to later chapters):**

**Marker**

TODO: Add a small mapping table from issues → requirements → implementation mechanisms (one row per requirement).

TODO: Decide which requirements are *must-have* vs. *nice-to-have* for the first version of Assertify.

TODO: State explicitly which requirements are out of scope (e.g., improving compiler diagnostics beyond test reporting).

### 3.3.3 Completion Checklist for This Chapter

**Marker**

- Add at least one fully worked example for each failure category (compiler error, unit test failure, property-based failure).
- Add an explicit "signal vs. noise" annotation for each example output.
- Ensure every derived requirement is supported by at least one observed issue (traceability).
- Add short, properly cited connections to prior work on diagnostic usability and novice comprehension (do not overclaim; separate observation from generalisation).
- Verify terminology consistency with later chapters (e.g., always use *tested expression* / *expected* / *actual* in the same order).

# 4 Assertify (and Checkify): Design and Implementation

**Marker**

*IMPROVE*

This chapter presents the design rationale and technical realisation of *Assertify* and *Checkify*, a lightweight library intended to improve the pedagogical usefulness of test feedback for novice programmers. The starting point is the requirement set derived in Problem Analysis. Students need concise, consistently structured reports that make the tested expression and the expected/actual mismatch explicit, while avoiding the fragmentation and verbosity typical of baseline frameworks. The overall approach is *non-invasive*: rather than replacing existing test runners, Assertify/Checkify wrap established tooling and refine the reported information before failing the test. [3], [2]

## 4.1 Design Goals and Principles

**Marker**

*IMPROVE*

The design of Assertify is guided by four principles that target novice-relevant shortcomings of baseline output: clarity, contextualisation, usability, and flexibility. These principles mirror the objectives stated for the library in the current thesis draft, but are restated here in a form that directly motivates concrete technical decisions.

**Clarity through a stable report schema.** The primary goal is to present failure information in a structured and easily readable form, such that learners can quickly identify (i) what was evaluated, (ii) what was expected, and (iii) what was actually obtained. In contrast to baseline failures that interleave this information with low-level noise, Assertify prioritises a compact summary view centred on an explicit expected/actual comparison.

**Context where it supports debugging.** Some failures cannot be understood solely from a final mismatch, especially when a failure is preceded by intermediate evaluations or when expressions are syntactically complex. Assertify therefore supports adding context in two complementary ways: (i) simplifying or reducing expressions to remove irrelevant technical detail while preserving the structure

that matters for interpretation, and (ii) optionally attaching a *history* of previously evaluated expressions in scenarios where a sequence of operations precedes a failure (e.g., stateful data structure tests).

**Usability and minimal intrusion.** A central design choice is to integrate into existing workflows without requiring a new execution environment or substantial changes to test code. The library is therefore implemented as a wrapper around established frameworks (e.g., MSTest assertions and FsCheck properties) and aims to preserve familiar testing idioms while improving the resulting feedback.

**Flexibility and configuration.** Students, tutors, and developers can have different information needs. Assertify is therefore designed to support configurable verbosity, for example by enabling or disabling optional report components such as reduction steps or history information. This allows the same testing infrastructure to serve both novice-oriented feedback (minimal, high-signal summaries) and more detailed debugging scenarios (extended context).

## 4.2 Output Model: Representing Tests

**Marker**

*IMPROVE*

To make failure feedback consistent across different testing mechanisms, Assertify/Checkify use a common *output model* that represents the essential components of a test outcome in a structured form. The model is designed around the minimal information that is directly actionable for novices: the tested expression and an explicit expected/actual comparison. This directly operationalises the requirements derived from the problem analysis chapter.

**Core fields.** At the centre of the model are three fields:

1. **Tested expression**: a textual representation of what was evaluated (e.g., `digitSum 123N = 6N`), intended to be readable without requiring knowledge of the test harness.

2. **Expected value**: the reference value implied by the specification or by the right-hand side of an equality.

3. **Actual value**: the value produced by evaluating the implementation under test.

This triad supports the stable reading strategy targeted by Assertify: identify the expression, compare expected vs. actual, then localise the relevant part of the implementation.

**Readable vs. technical renderings.** In practice, expression strings produced by tooling may include module qualifications or syntactic artifacts that are not helpful for novices. The model therefore distinguishes between a *full* expression rendering (useful for traceability in the pipeline) and an optional *simplified* rendering intended for student-facing output. This design is reflected in the before/after examples already included in the draft, where both the fully qualified and simplified versions are displayed.

**Optional context fields.** The model also supports optional context fields that may be included depending on the test mechanism and configuration:

- **History**: a sequence of successfully evaluated expressions leading up to a failure, intended to preserve operational context in multi-step tests.
- **Property-based inputs**: original and shrunk counterexamples for property-based failures, enabling students to focus on the minimal failing case while retaining access to the unshrunk case for additional context.
- **Diagnostic metadata**: a message and stack trace for traceability and for tutor/debugging use, while keeping this information secondary to the student-facing summary.

**Serialisation and downstream use.** A structured output model also enables alternative presentations beyond plain console text. The project notes explicitly mention serialisable outputs (e.g., JSON) as a basis for visualisation and for integrating reports into external tooling (such as IDE support). While the thesis primarily discusses the human-readable rendering, the model is designed to remain usable in such downstream scenarios.

## 4.3 Mechanisms: Expression Capture and Value Extraction

The output model introduced in the previous section centers on three core fields: (i) a *tested expression* that communicates what was evaluated in a harness-independent form — without needing to know anything about the surrounding test infrastructure, (ii) an *expected value* that reflects the specification, and (iii) an *actual value* obtained from executing the implementation under test. Together, these fields shift the focus of test feedback from *how* a test is implemented to *what* claim the test makes about the student program.

However, standard unit testing APIs typically observe only the *outcome* of an assertion — for instance, two already-computed values passed into an `AreEqual` check, or a boolean predicate. The literal source-level claim that the test author wrote is not preserved. Reconstructing the *tested expression* field therefore cannot be achieved by merely forwarding existing assertion parameters. Providing the full test harness would reintroduce the very problem the model aims to avoid, namely

that students must read and interpret auxiliary test code instead of focusing on the semantic mismatch.

A practical observation is that many example-based tests and a large subset of property-based tests can be normalized to an equality-shaped claim of the form *actual* = *expected*, even if the surrounding test method constructs inputs, performs intermediate computations, or more complex tasks. Consequently, the key technical requirement is to capture the *claim itself* in a form that can later be displayed and analyzed, and then to evaluate that representation to obtain the expected and actual values for reporting.

### 4.3.1 Expression capture via code quotations

F# offers code quotations as a structured representation of code that can be inspected and evaluated later.

**Code quotations:** a language feature that enables you to generate and work with F# code expressions programmatically. This feature lets you generate an abstract syntax tree that represents F# code. The abstract syntax tree can then be traversed and processed according to the needs of your application. For example, you can use the tree to generate F# code or generate code in some other language. [1]

A simple example for a code quotation is `let expr: Expr<int> = <@ 1 @>` with `1` being the code to be represented, wrapped by the symbols `<@` and `@>`. The code quotation is stored using the non-generic type `Expr`. [6] Consider the following example expression:

```
let f = <@ fun (a: int) (b: int) -> a + b @>
```

**Listing 4.1** Example code quotation

**Characterization of represented code.** The code inbetween can be characterized by a few things: (i) it is an anonymous function or *lambda* expression, (ii) it is semantically equivalent to `fun (a: int) -> (b: int) -> a + b` and (iii) with that we can also see the explicit type of this lambda: `int -> int -> int`.

**Characterization of expression.** A look at the string representation reveals that we are dealing with a nested lambda expression of form:

```
Lambda (a, Lambda (b, Call (None, op_Addition, [a, b])))
```

which (i) seems to align with the pattern of the original code — the types are not printed yet present so type information is not lost, (ii) seems to be semantically equivalent to the represented code and lastly (iii) type inference implies the type `Expr<int -> int -> int>`.

In conclusion, the comparison shows that `fun (a:int) (b:int) -> a + b` denotes a curried function, whereas `<@ fun (a:int) (b:int) -> a + b @>` denotes a quotation of that function expression. Accordingly, the two expressions are not equivalent at the level of F# evaluation: the former evaluates to an executable function value, while the latter evaluates to a representation of the function's syntax. However, the quotation still captures a term with the same computational behavior as the unquoted function and since preserving syntactic structure its crucial advantage is enabling inspection and transformation of the program as data. At the same time, quotations remain executable in the sense that they can be evaluated (or partially evaluated) programmatically and — where required — transformed back into executable function values. Thus, quotations unite syntactic preservation with computational expressiveness.

The `Microsoft.FSharp.Quotations` namespace offers various active patterns to analyze expressions, as well as a member `Substitute` from the `Expr` type to map variables to new values, which is helpful for partial evaluations of curried functions.

**Swensen.Unquote library.** Before going into more detail about the value extraction, it is worth pointing out that there are already libraries offering testing tools using code quotations like Swensen.Unquote. [4] The following example using Unquote's `test` method illustrates how a quoted equality implicitly contains all three core fields — tested expression, expected value, and actual value:

```
let add a b = a + b + 1
test <@ add 1 2 = 1 + 2 @>
```

```
Swensen.Unquote.AssertionFailedException:  Test failed:

add 1 2  =  1 + 2
4  =  3
false
```

**Listing 4.2** Example `Swensen.Unquote.test` output

In blue the tested expression `add 1 2`, in red the actual value and in green the expected value. Combining code quotations and Swensen.Unquote offers not only functionality to analyze code via known patterns, but also to easily modify it via functionalities like: `eval`, `reduce`, `decompile`, `substitute` or the above mentioned `test`. [6], [5]

While this output makes the mismatch transparent, it also hints at a design tension: depending on the level of detail, the output may reveal how the expected value is constructed (here: the right-hand side expression), which can unintentionally disclose test-oracle information. In orange it is visible that the correct computation seems to be `1 + 2`, which strongly suggest the correct implementation `let add a b = a + b`. Depending on the usage, information may leak that is not intended for students.

While offering a good foundation to suit the usecase some optimization are needed which I will mention later.

### 4.3.2 Value extraction for unit tests and property based tests

While quotations preserve the structure of a test claim, Assertify still needs to *separate* the claim into the parts that correspond to the output model (i.e., tested expression, expected value, and actual value) and then *evaluate* those parts. This is achieved by restricting the supported shape of claims to a normal form that is both common in introductory exercises and directly compatible with the reporting scheme: an equality-shaped claim of the form *actual* = *expected*.

In practice, such claims are represented as quoted boolean expressions. At runtime, Assertify and Checkify inspect these quotations using standard quotation patterns. In the intended case, the quotation contains a call to the equality operator, which can be deconstructed into its left-hand side (the implementation/result under test) and right-hand side (the reference implementation or result). This deconstruction step forms the bridge from quoted code to the three-field output model: the *tested expression* is rendered from the left-hand side (with concrete inputs substituted, if necessary), while *actual* and *expected* are obtained by evaluating the left and right subexpressions respectively.

**Unit tests.** For ordinary unit tests, the quoted claim already contains concrete values. Accordingly, extraction proceeds directly by deconstructing the equality-shaped quotation into its two subexpressions and evaluating them independently. Conceptually, this maps

$$\texttt{actualExpr} = \texttt{add 1 2} \qquad \texttt{expectedExpr} = \texttt{1 + 2}$$
$$[\![\texttt{actualExpr}]\!] = \texttt{4} \qquad [\![\texttt{expectedExpr}]\!] = \texttt{3}$$

where $[\![\cdot]\!]$ denotes evaluation of a quotation.

This also ensures that no solution-relevant information is leaked to students in raw syntactic form. For example, in a claim such as `add 1 2 = 1 + 2`, the critical right-hand side is not exposed as source code in the feedback, but only through its evaluated result. This supports the didactic goal of revealing enough information to diagnose the error while avoiding direct disclosure of the reference solution.

This also assures that no information is leaked to students, because the critical right hand side `1 + 2` in 4.2 would only be visible to the student in evaluated form.

With that a stable basis for reporting is provided, because it avoids reliance on ad-hoc parsing of framework error strings and instead operates on the structured representation of the asserted claim. If the quotation does not match the expected equality pattern, the mechanism cannot reliably derive *expected* and *actual* values at this point of implementation. In that case, Assertify falls back to reporting the captured expression and a generic failure message. This limitation is deliberate: extending support to arbitrary assertion forms is possible, but requires separate, assertion-specific deconstruction rules (e.g., inequalities, exception checks, boolean predicates).

**Algorithm 1** Assertify.Test

```
let Test(expr: Expr<bool>, ?message: string)
    if message = null then
        message ← defaultMsg
    try
        if ⟦ expr ⟧ then
            return success
        else
            return ConstructErrorMsg(expr, message)
    with
        return DefaultErrorMsg(expr, message)
```

**Algorithm 2** Checkify.Check

```
let Check(expr: Expr, check: Expr -> 'Testable)
    capture runner
    try
        FsCheck.Check (check expr)                ▷ Successful test
    return success
    with Hallo
        try add 1 2
            Test 1 2 3
```



**Marker**

*IMPROVE*

$$actualValue = [actualExpr] \qquad \text{and} \qquad expectedValue = [expectedExpr]$$

where $[\cdot]$ denotes evaluation of a quotation. In the implementation, evaluation is performed using the quotation evaluation facilities provided by the F# runtime and by Swensen.Unquote.

A practical complication is that evaluation may itself fail. This happens, for example, when the student implementation throws an exception, when a branch contains a remaining placeholder such as `failwith "TODO"`, or when a quoted expression cannot be evaluated due to missing runtime context. For novice feedback, a raw evaluation exception would reintroduce the problem of unstructured output. Assertify therefore treats evaluation failure as a first-class case and produces a controlled placeholder (e.g., "non-evaluatable") rather than propagating the exception as the primary message. The full technical details of these fallbacks are discussed later with the concrete helper functions.

### Example-based tests: closed expressions

In example-based unit tests, the quoted claim is typically *closed*: all inputs are written explicitly in the test case (e.g., `digitSum 123N = 6N`). This means that value extraction can proceed immediately after capture: deconstruct the equality, evaluate both sides, and render the tested expression.

For reporting, Assertify distinguishes two closely related representations: (i) a direct rendering of the left-hand side as the tested expression (to answer "what was executed?") and (ii) the evaluated values of both sides (to answer "what was expected and what was produced?"). In this setting, no additional context is required beyond the captured quotation itself.

### Property-based tests: open expressions and captured inputs

Property-based tests differ in one crucial aspect: the claim is generally *open* because it depends on generated inputs. Instead of quoting a closed boolean, a property-based test typically captures a quoted lambda expression, such as `<@ fun n -> sortedDigits n = sortedDigitsSolution n @>`. The quotation therefore describes the *shape* of the claim, but evaluation and rendering require concrete input values.

FsCheck provides these values only when a counterexample is found. In addition, FsCheck may *shrink* a failing input to a simpler one. For the purposes of novice-oriented feedback, the most relevant inputs are therefore the final counterexample values after shrinking, because they are the smallest (and usually easiest to reason about) values that still falsify the property.

### Runner-assisted extraction with `CaptureRunner`

To make counterexample inputs available to the reporting layer, Checkify wraps the configured FsCheck runner with a custom runner (`CaptureRunner`). The

runner intercepts the test result at the end of execution. When FsCheck reports a failure, the runner stores both the original failing arguments and the final shrunk arguments.

The stored inputs are then used in two places:

1. **Instantiation for rendering.** The open quotation (a lambda) is instantiated by substituting the captured counterexample values for its parameters. This yields a quotation that corresponds to the concrete failing call, which can be rendered as the *tested expression* shown to the student.

2. **Instantiation for evaluation.** The same substitution step yields a concrete equality claim whose left and right subexpressions can be deconstructed and evaluated to obtain *actual* and *expected*.

In Checkify, this instantiation is implemented as iterative application of arguments to a quoted lambda, using quotation substitution. The application function traverses nested lambdas and replaces parameter occurrences with `Expr.Value` nodes constructed from the captured objects. This keeps the mechanism uniform across different arities and preserves the key invariant: *the displayed tested expression corresponds to the failing counterexample that FsCheck reports*.

#### Reduction traces: diagnostic benefit vs. oracle leakage

Some quotation-based tools provide additional context by showing a reduction trace, i.e., intermediate simplifications of an expression until a final form is reached. This can make the connection between a complex claim and its final mismatch more explicit, particularly for novices who struggle to mentally execute expressions.

At the same time, reduction traces can reveal how an expected value was constructed (for instance by exposing the full right-hand side computation), which may disclose test-oracle information. Since Assertify targets an exercise-oriented setting, it treats such detail as optional and subject to verbosity control rather than as a mandatory default. This makes it possible to trade diagnostic transparency against pedagogical constraints depending on the context in which the feedback is used.

**Listing 4.1:** *Example of a reduction trace (adapted from the Unquote user guide)*

```
Test failed:

(1 + 2) / 3 = 2
3 / 3 = 2
1 = 2
false
```

**Summary**

Expression capture and value extraction are the technical prerequisites for producing structured test feedback in the form of *tested expression*, *expected value*, and *actual value*. Quotations enable the capture of asserted claims as structured data; quotation patterns allow the system to identify and deconstruct equality-shaped claims; and evaluation of the resulting subexpressions yields the runtime values required for reporting.

For property-based testing, quotations alone are not sufficient because claims are open with respect to generated inputs. Checkify therefore integrates at the runner level: `CaptureRunner` records the original and shrunk counterexample arguments so that the captured quotation can be instantiated, rendered, and evaluated in the concrete context of the failing assignment. The implementation details of these helper functions and their formatting choices are covered later in this chapter.

## 4.4 Reporting: Formatting and Integration with the Test Pipeline

**Marker**

*IMPROVE*

Assertify/Checkify are designed to integrate into existing test workflows by wrapping standard frameworks and replacing baseline failure text with a structured report. The integration strategy follows the "minimal intrusion" principle: tests are still executed by conventional runners, but failures are intercepted and re-raised with enriched information in a stable format.

### 4.4.1 Integration with example-based unit tests

For example-based tests, Assertify acts as a wrapper around standard assertion-style checks by evaluating the quoted test expression and failing the test if it evaluates to `false`. When a failure occurs, Assertify produces a report that explicitly includes:

- the tested/failed expression,
- an optional simplified expression rendering, and
- the expected and actual values.

This structure is illustrated by the existing before/after comparison listings in the draft, where Assertify reports the fully qualified expression alongside a simplified rendering and then prints expected and actual results in a dedicated block.

### 4.4.2 Integration with property-based tests

For property-based tests, Checkify provides a wrapper that executes the property via FsCheck and, on failure, captures both the original and shrunk counterexample inputs. These inputs are incorporated into the structured report to make the falsifying case reproducible and to ensure that the student-facing expression is presented with the relevant (typically shrunk) input instantiated. This preserves the value of shrinking while presenting the result in the same expected/actual-oriented format as example-based tests.

### 4.4.3 Formatting strategy

The reporting format is designed around a high-signal summary that foregrounds the novice-relevant fields. The draft output examples show explicit section markers (e.g., for "TESTED EXPRESSION" and "EXPECTED AND ACTUAL"), which support scanning and reduce the likelihood that students miss the crucial mismatch due to stack trace noise. Additional details such as stack traces remain available for traceability but are treated as secondary information.

### 4.4.4 Configurable verbosity and optional context

Finally, reporting is configurable with respect to optional context components. In line with the stated design considerations, the library supports toggling whether reductions and history information are included, allowing the same testing setup to be used with different levels of verbosity depending on the educational context (introductory debugging vs. more advanced troubleshooting).

# 5 Results and Discussion

**Marker**

- Technical validation: what works, coverage, limitations
- Structured comparison: criteria-based analysis, referencing earlier examples

## 5.1 Research Question Coverage and Requirement Satisfaction

**Marker**

*TODO*

## 5.2 Artifact Outcomes

**Marker**

*TODO*

## 5.3 Key Findings and Interpretations

**Marker**

*TODO*

## 5.4 Strengths and Limitations

**Marker**

*TODO*

# 6 Conclusion

**Marker**

*TODO*

## 6.1 Summary

**Marker**

*TODO*

## 6.2 Future Work

**Marker**

- Optimizations
- Useful Evaluation/Data once the code is running smoothly on ExClaim
- FSharp.Compiler.Services to reflect on what went wrong:
- A combination of FSharp.Compiler.Services and FParsec would be able to produce better errors for majority of failures
- TODO DATA OF TOP 10 ERROR CODES !!!!!
- Generalization via MiniLib:
- Including student friendly helper modules/types/etc
- Test Library with configurable verbosity to suit multiple levels of learning progress
- Maybe implement it as a full Test Framework
- Combine with VS Code and code-server?

## 6.3 Final Remarks

**Marker**

*TODO*

# List of Figures

# List of Listings

4.1 Example of a reduction trace (adapted from the Unquote user guide) 23

# List of Tables

# My Code

**Marker**

*TODO*

# My Ideas

**Marker**

I have no idea what I'm doing.

# Bibliography


[1] *.NET Documentation: Code quotations*. https://learn.microsoft.com/en-us/dotnet/fsharp/language-reference/code-quotations.

[2] Titus Barik, Justin Smith, Kevin Lubick, Elisabeth Holmes, Jing Feng, Emerson Murphy-Hill, and Chris Parnin. "Do Developers Read Compiler Error Messages?" In: *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*. 2017, pp. 575–585. DOI: 10.1109/ICSE.2017.59.

[3] Brett A. Becker, Paul Denny, Raymond Pettit, Durell Bouchard, et al. "Compiler Error Messages Considered Unhelpful: The Landscape of Text-Based Programming Error Message Research". In: *Proceedings of the Working Group Reports on Innovation and Technology in Computer Science Education*. ITiCSE-WGR '19. Aberdeen, Scotland Uk: Association for Computing Machinery, 2019, pp. 177–210. ISBN: 9781450375672. DOI: 10.1145/3344429.3372508. URL: https://doi.org/10.1145/3344429.3372508.

[4] *Github: Swensen.Unquote*. https://github.com/SwensenSoftware/unquote.

[5] *Github: Swensen.Unquote User Guide*. https://github.com/SwensenSoftware/unquote/wiki/UserGuide.

[6] *Github.io: Expr Type*. https://fsharp.github.io/fsharp-core-docs/reference/fsharp-quotations-fsharpexpr.html.